MODUL 13 · BAB V · ALGORITMA GREEDY

Konsep Algoritma Greedy

Memilih yang terlihat terbaik saat ini: kapan strategi sederhana ini menghasilkan solusi optimal, dan kapan gagal.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menjelaskan prinsip kerja algoritma Greedy: pilihan optimal lokal dengan harapan menghasilkan optimal global.
- Menyebutkan dua sifat yang menjamin Greedy optimal: *greedy-choice property* dan *optimal substructure*.
- Membedakan Greedy dari Dynamic Programming dari cara mengambil keputusan.
- Menunjukkan dengan contoh penyangkal bahwa Greedy tidak selalu optimal.
- Mengenali pilihan greedy pada algoritma-algoritma di modul selanjutnya.

PRASYARAT

- Modul 10: konsep Dynamic Programming dan masalah optimasi.

BAB / MATERI ASAL

Materi 10.1 ALGORITMA GREEDY An Activity Selection Problem (bagian pengantar Algoritma Greedy)

POSISI DALAM URUTAN BELAJAR

Konsep umum Bab V. Diterapkan pada Activity Selection (Modul 14) dan Huffman (Modul 15), serta pada MST (Modul 19–21) dan Dijkstra (Modul 24).

DAFTAR ISI

1. Ketika Mencoba Semua Pilihan Terasa Berlebihan 1
2. Prinsip Algoritma Greedy 2
3. Kapan Greedy Menghasilkan Solusi Optimal? 2
4. Contoh: Ketika Greedy Berhasil dan Gagal 3
5. Peta Algoritma Greedy dalam Mata Kuliah Ini 5
6. Kesimpulan 6
- ◆ Kuis Pemahaman 7
- ◆ Kunci Jawaban dan Pembahasan 8
- ◆ Referensi 8

PENDAHULUAN

1. Ketika Mencoba Semua Pilihan Terasa Berlebihan

Algoritma untuk masalah optimasi biasanya berjalan melalui serangkaian langkah, dan di setiap langkah ada beberapa pilihan. Dynamic Programming (Modul 10–12) menangani pilihan-pilihan itu dengan sangat hati-hati: ia mencoba **semua** pilihan, menyelesaikan submasalah untuk masing-masing, lalu baru memutuskan mana yang terbaik.

Untuk banyak masalah optimasi, kehati-hatian seperti itu ternyata berlebihan. Ada strategi yang jauh lebih sederhana dan efisien: **di setiap langkah, ambil saja pilihan yang terlihat paling baik saat itu**, tanpa memikirkan akibatnya nanti dan tanpa pernah meninjau ulang keputusan. Strategi ini disebut **algoritma Greedy** (rakus), karena ia selalu "mengambil yang terbesar/terbaik sekarang".

Modul ini membahas konsep Greedy secara umum: cara kerjanya, syarat agar ia menghasilkan solusi optimal, contoh-contoh ketika ia gagal, dan peta algoritma Greedy yang akan kita pelajari di modul-modul berikutnya.

PEMBAHASAN INTI

2. Prinsip Algoritma Greedy

DEFINISI

Algoritma Greedy selalu membuat pilihan yang terlihat paling baik pada saat itu. Artinya, pilihan yang dilakukan adalah pilihan yang **optimal secara lokal**, dengan harapan rangkaian pilihan tersebut akan menghasilkan solusi yang **optimal secara global**.

Pola umum algoritma Greedy:

1. Tentukan **kriteria greedy**: ukuran "terbaik saat ini", misalnya waktu selesai paling awal, frekuensi paling kecil, atau bobot paling ringan.
2. Ambil kandidat terbaik menurut kriteria itu, selama tidak melanggar batasan masalah.
3. Masukkan kandidat ke solusi. Keputusan ini **tidak pernah dibatalkan**.
4. Ulangi pada masalah yang tersisa sampai tidak ada kandidat lagi.

Karena setiap langkah hanya membuat satu keputusan dan tidak pernah mundur, algoritma Greedy biasanya sederhana dan cepat. Waktu yang dominan sering kali hanya untuk mengurutkan kandidat menurut kriteria greedy, yaitu $O(n \lg n)$.

PENTING

Bahan kuliah menegaskan: **algoritma Greedy tidak selalu menghasilkan solusi yang optimal.** Meskipun demikian, Greedy cukup kuat dan bekerja dengan baik untuk berbagai masalah, misalnya kompresi data dengan pengkodean Huffman, *activity-selection problem*, *minimum spanning tree*, dan *single-source shortest path* dengan algoritma Dijkstra. Untuk setiap masalah, kebenaran algoritma Greedy harus **dibuktikan**, tidak boleh diandaikan.

PEMBAHASAN INTI

3. Kapan Greedy Menghasilkan Solusi Optimal?

Cormen dkk. (2009, Bab 16.2) menyebutkan dua sifat yang, bila dimiliki suatu masalah, menjamin algoritma Greedy yang sesuai menghasilkan solusi optimal.

3.1 Greedy-choice property (sifat pilihan greedy)

Selalu ada solusi optimal yang memuat pilihan greedy pertama. Dengan kata lain, mengambil pilihan yang terlihat terbaik saat ini **tidak menutup jalan** menuju solusi optimal. Inilah sifat yang membedakan masalah Greedy dari masalah Dynamic Programming biasa.

Cara pembuktian yang paling umum adalah **argumen pertukaran** (*exchange argument*): ambil sembarang solusi optimal; jika tidak memuat pilihan greedy, tukar salah satu elemennya dengan pilihan greedy dan

tunjukkan bahwa solusinya tetap sah dan nilainya tidak memburuk. Contohnya akan kita lihat pada Activity Selection di Modul 14.

3.2 Optimal substructure

Setelah pilihan greedy dibuat, yang tersisa adalah **satu** submasalah yang bentuknya sama, dan solusi optimal keseluruhan = pilihan greedy + solusi optimal submasalah itu. Sifat ini sama dengan yang dibutuhkan Dynamic Programming (Modul 10).

3.3 Greedy dibandingkan dengan Dynamic Programming

Aspek	Dynamic Programming	Greedy
Kapan memilih	Setelah semua submasalah yang relevan diselesaikan	Sebelum menyelesaikan submasalah
Banyaknya pilihan yang dievaluasi	Semua pilihan	Hanya satu, yang terlihat terbaik
Arah penyelesaian	Biasanya bottom-up (dari submasalah terkecil)	Top-down: pilih, lalu kecilkan masalah
Submasalah tersisa	Banyak, saling tumpang tindih, disimpan di tabel	Satu
Jaminan optimal	Selalu, bila ada optimal substructure	Hanya bila ada greedy-choice property
Efisiensi	Polinomial, sering $\Theta(n^2)$ atau $\Theta(n^3)$	Sering $O(n \lg n)$ atau lebih cepat

Tabel 1. Perbedaan cara mengambil keputusan.

Bahan kuliah menyebutkan bahwa masalah activity selection dapat diselesaikan dengan Dynamic Programming maupun Greedy. Karena masalah itu memiliki greedy-choice property, cara Greedy jauh lebih sederhana dan efisien.

CONTOH DAN PENERAPAN

4. Contoh: Ketika Greedy Berhasil dan Gagal

4.1 Contoh 1: menukar uang dengan koin paling sedikit

Kriteria greedy: ambil koin dengan nilai **terbesar** yang tidak melebihi sisa uang.

- Pecahan $\{1, 5, 10, 25\}$, jumlah 63: greedy memilih $25 + 25 + 10 + 1 + 1 + 1$ (6 koin). Ini optimal.
- Pecahan $\{1, 3, 4\}$, jumlah 6: greedy memilih $4 + 1 + 1$ (3 koin), padahal $3 + 3$ hanya **2 koin**. **Greedy gagal.** Mengambil koin 4 terlihat terbaik saat itu, tetapi menutup jalan menuju solusi optimal: sistem pecahan ini tidak memiliki greedy-choice property.

4.2 Contoh 2: knapsack pecahan dan knapsack 0-1

Contoh dari Cormen dkk. (2009, Bab 16.2). Sebuah tas berkapasitas 50 kg. Tersedia tiga barang:

Barang	Berat (kg)	Nilai	Nilai per kg
1	10	60	6
2	20	100	5
3	30	120	4

Tabel 2. Barang yang tersedia.

Kriteria greedy: ambil barang dengan **nilai per kg terbesar** lebih dulu.

- **Knapsack pecahan** (barang boleh diambil sebagian): ambil seluruh barang 1 (60) dan barang 2 (100), lalu $\frac{20}{30}$ barang 3 (80). Total 240, dan ini **optimal**.
- **Knapsack 0-1** (barang diambil utuh atau tidak sama sekali): greedy mengambil barang 1 dan 2 (total 160, berat 30), lalu barang 3 tidak muat. Solusi optimalnya justru barang 2 dan 3, total **220**. **Greedy gagal**; masalah ini harus diselesaikan dengan Dynamic Programming.

Pelajaran pentingnya: dua masalah yang terlihat hampir sama bisa berbeda dalam hal apakah Greedy berlaku.

4.3 Contoh 3: memilih kriteria greedy yang tepat

Pada masalah activity selection (Modul 14), bahan kuliah mencoba tiga kriteria berbeda pada data yang sama. Kriteria "waktu mulai paling awal" hanya menghasilkan 3 aktivitas, padahal optimalnya 4. Kriteria "waktu selesai paling awal" selalu optimal. Jadi untuk satu masalah yang sama, sebagian kriteria greedy benar dan sebagian salah.

4.4 Memeriksa dengan program

Python: greedy vs solusi optimal pada masalah koin dan knapsack

```python
from itertools import combinations

def koin_greedy(pecahan, x):
    hasil = []
    for k in sorted(pecahan, reverse=True):
        while x >= k:
            hasil.append(k)
            x -= k
    return hasil

def koin_dp(pecahan, x):                  # Dynamic Programming: koin minimum
    terbaik = [0] + [float("inf")] * x
    for v in range(1, x + 1):
        terbaik[v] = min(terbaik[v - k] + 1 for k in pecahan if k <= v)
    return terbaik[x]

print(koin_greedy([1, 5, 10, 25], 63), koin_dp([1, 5, 10, 25], 63))
print(koin_greedy([1, 3, 4], 6), koin_dp([1, 3, 4], 6))

barang = [(10, 60), (20, 100), (30, 120)]    # (berat, nilai)
W, total, berat = 50, 0, 0
for b, v in sorted(barang, key=lambda t: t[1] / t[0], reverse=True):
    if berat + b <= W:                        # greedy 0-1
        berat, total = berat + b, total + v
optimal = max(sum(v for _, v in s) for r in range(4)
              for s in combinations(barang, r) if sum(b for b, _ in s) <= W)
print("greedy 0-1:", total, " optimal 0-1:", optimal)
```

Keluaran program

```
[25, 25, 10, 1, 1, 1] 6
[4, 1, 1] 2
greedy 0-1: 160  optimal 0-1: 220
```

ANALISIS DAN HUBUNGAN KONSEP

5. Peta Algoritma Greedy dalam Mata Kuliah Ini

Bahan kuliah menyebutkan empat penerapan Greedy yang semuanya terbukti optimal. Masing-masing memiliki "pilihan greedy" yang khas:

Algoritma	Masalah	Pilihan greedy di setiap langkah	Modul
GREEDY-ACTIVITY-SELECTOR	Memilih aktivitas sebanyak mungkin yang tidak bertumpang tindih	Aktivitas kompatibel dengan waktu selesai **paling awal**	14
Huffman	Kode prefiks dengan total bit minimum	Gabungkan dua simpul dengan frekuensi **paling kecil**	15
Kruskal	Minimum spanning tree	Busur **paling ringan** yang tidak membentuk cycle	20
Prim	Minimum spanning tree	Busur **paling ringan** yang menghubungkan pohon dengan simpul baru	21
Dijkstra	Lintasan terpendek dari satu sumber (bobot nonnegatif)	Simpul dengan jarak sementara **paling kecil**	24

Tabel 3. Pilihan greedy pada algoritma-algoritma berikutnya.

Mengapa algoritma ini benar? Setiap masalah di tabel memiliki greedy-choice property, yang dibuktikan dengan argumen pertukaran (Activity Selection, Huffman), sifat potongan graf (Kruskal, Prim; Modul 19), atau invarian jarak (Dijkstra). Kebenaran Dijkstra bahkan bergantung pada syarat tambahan, yaitu tidak ada bobot negatif; Modul 24 menunjukkan contoh ketika Dijkstra salah bila syarat itu dilanggar.

Strategi umum ketika menghadapi masalah baru. Coba temukan kriteria greedy yang masuk akal, lalu cari contoh penyangkal kecil. Jika contoh penyangkal ditemukan (seperti koin $\{1, 3, 4\}$), beralihlah ke Dynamic Programming. Jika tidak ditemukan, cobalah membuktikan greedy-choice property dengan argumen pertukaran.

KESIMPULAN

6. Kesimpulan

- Algoritma Greedy membuat pilihan optimal lokal di setiap langkah dan tidak pernah meninjau ulang keputusannya.
- Greedy menghasilkan solusi optimal bila masalahnya memiliki *greedy-choice property* dan *optimal substructure*.
- Berbeda dengan Dynamic Programming yang mengevaluasi semua pilihan, Greedy memilih lebih dulu lalu menyelesaikan satu submasalah yang tersisa, sehingga biasanya lebih sederhana dan cepat.
- Greedy tidak selalu optimal: contoh penyangkalnya antara lain koin $\{1, 3, 4\}$ dan knapsack 0-1.
- Algoritma Greedy yang terbukti optimal dalam mata kuliah ini: Activity Selection, Huffman, Kruskal, Prim, dan Dijkstra.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 PILIHAN GANDA

Ciri utama algoritma Greedy adalah ...

A. Mencoba semua kemungkinan solusi lalu memilih yang terbaik.
B. Menyimpan solusi submasalah di tabel.
C. Memilih yang terlihat terbaik saat ini tanpa meninjau ulang keputusan.
D. Selalu menghasilkan solusi optimal.

2 PERHITUNGAN

Dengan pecahan koin $\{1, 6, 10\}$, tentukan hasil greedy untuk jumlah 12. Apakah optimal? Jika tidak, berapa jumlah koin minimal?

3 BENAR/SALAH DENGAN ALASAN

"Jika sebuah algoritma Greedy memberikan jawaban optimal untuk beberapa contoh data, maka algoritma itu terbukti benar." Benar atau salah?

4 ANALISIS

Jelaskan perbedaan cara Dynamic Programming dan Greedy dalam mengambil keputusan, menggunakan istilah "sebelum" dan "sesudah" menyelesaikan submasalah.

5 ANALISIS KASUS

Pada contoh knapsack di modul ini, kapasitas tas diubah menjadi 30 kg. Hitung hasil greedy (berdasarkan nilai per kg) untuk knapsack 0-1 dan bandingkan dengan solusi optimalnya.

6 PILIHAN GANDA

Sifat yang menyatakan "selalu ada solusi optimal yang memuat pilihan greedy pertama" disebut ...

A. Optimal substructure
B. Overlapping subproblems
C. Greedy-choice property
D. Loop invariant

7 ANALISIS

Untuk algoritma Dijkstra dan algoritma Huffman, sebutkan pilihan greedy yang dibuat di setiap langkah.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: C. Pilihan A adalah brute force, B adalah Dynamic Programming, dan D salah karena Greedy tidak selalu optimal.

Soal 2

Greedy: $10 + 1 + 1$ = **3 koin**. Tidak optimal, karena $6 + 6$ hanya **2 koin**.

Soal 3

Salah. Beberapa contoh yang berhasil tidak membuktikan apa pun; cukup satu contoh penyangkal untuk menunjukkan algoritma salah (seperti koin $\{1, 3, 4\}$ yang berhasil untuk banyak jumlah, tetapi gagal untuk 6). Kebenaran harus dibuktikan, misalnya dengan argumen pertukaran.

Soal 4

Dynamic Programming memilih **sesudah** menyelesaikan submasalah: nilai setiap pilihan bergantung pada solusi submasalah yang sudah dihitung, lalu dipilih yang terbaik. Greedy memilih **sebelum** menyelesaikan submasalah: pilihan diambil berdasarkan informasi saat itu, baru kemudian satu submasalah yang tersisa diselesaikan.

Soal 5

Urutan nilai per kg: barang 1 (6), 2 (5), 3 (4). Greedy: barang 1 (10 kg, 60), barang 2 (20 kg, 100), total berat 30 kg; barang 3 tidak muat. Total **160**. Kemungkinan lain yang muat: barang 3 saja (120), barang 1 + 2 (160). Jadi kali ini greedy **optimal** (160). Ini menunjukkan bahwa greedy kadang berhasil pada knapsack 0-1, tetapi tidak dapat diandalkan.

Soal 6

Jawaban: C. Optimal substructure (A) menyatakan solusi optimal tersusun dari solusi optimal submasalah.

Soal 7

Dijkstra: mengambil (extract) simpul yang jarak sementaranya dari sumber paling kecil di antara simpul yang belum selesai. **Huffman**: menggabungkan dua simpul (karakter atau subpohon) yang frekuensinya paling kecil menjadi satu simpul baru.

Referensi

Sumber utama: Materi 10.1 ALGORITMA GREEDY An Activity Selection Problem (bagian pengantar Algoritma Greedy) (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Boston: Pearson/Addison-Wesley.